BCSE204L Design and Analysis of Algorithms

Module-1

Topic-3: Greedy Techniques, Fractional Knapsack

Dr Sunil Kumar P V

SCOPE

Optimization Problems

- Maximization or minimization
- These problems have n inputs and require us to obtain a subset that satisfies some constraints
- Any subset that satisfies those constraints is called a feasible solution
- We need to find a feasible solution that either maximizes or minimizes a given objective function
- A feasible solution that does this is called an optimal solution
- Many feasible solutions but only one optimal solution
- Brute-force, greedy method, dynamic programming, branch and bound, all used to solve optimization problems

Greedy approach: working

```
1   Algorithm Greedy(a, n)
2   // a[1 : n] contains the n inputs.
3   {
4       solution := ∅; // Initialize the solution.
5       for i := 1 to n do
6       {
7           x := Select(a);
8           if Feasible(solution, x) then
9               solution := Union(solution, x);
10      }
11      return solution;
12  }
```

Greedy Examples

- Prim’s
- Kruskal’s
- Dijkstra’s
- Fractional knapsack
- Huffman coding

Knapsack Problem

- *Given a set of items, each with a weight and a profit (value), determine a subset of items to include in a collection so that the total weight is less than or equal to a given limit and the total profit is as large as possible.*

Applications

- Finding the least wasteful way to cut raw materials
- Portfolio optimization
- Cutting stock problems
- Transporting goods to sell over a different place
- Scenario
  - A thief is robbing a store and can carry a maximum weight of W into his knapsack. There are n items available in the store and weight of i^{th} item is w_i and its profit is p_i. What items should the thief take?
  - The objective of the thief is to maximize the profit.

Variations

1. Fractional knapsack problem
   - In this case, items can be broken into smaller pieces, hence the thief can select fractions of items
   - Greedy approach
2. 0/1 knapsack problem
   - The items cannot be divided. Either take fully or drop fully
   - Dynamic programming approach

- Here we will focus on fractional knapsack problem

Demo Example

Objects	1	2	3	4	5	6	7
Profits	10	5	15	7	6	18	3
Weights	2	3	5	7	1	4	1

Knapsack capacity, W=15

Requirements

- According to the problem statement,
  - There are n items in the store
  - Weight of i^{th} item, $w_i > 0$
  - Profit for i^{th} item, $p_i > 0$
  - Capacity of the knapsack is W
  - As items can be divided, $0 \leq x_i \leq 1$, where x_i is the fraction of item i included in the knapsack

Optimizing Knapsack problem

- Hence objective is
  - Maximize $\sum_{i=1}^{n} x_i p_i$
  - Subject to the constraint $\sum_{i=1}^{n} x_i w_i \leq W$
- At the optimal solution, $\sum_{i=1}^{n} x_i w_i = W$, otherwise we could add a fraction of one of the remaining items and increase the overall profit

Pseudocode

```
Algorithm: Greedy-Fractional-Knapsack (w[1..n], p[1..n], W)
for i = 1 to n
   do x[i] = 0
weight = 0
for i = 1 to n
   if weight + w[i] ≤ W then
      x[i] = 1
      weight = weight + w[i]
   else
      x[i] = (W - weight) / w[i]
      weight = W
      break
return x
```

Analysis

- If the provided items are already sorted into a decreasing order of p_i/w_i, then the while loop takes O(n) time
- Therefore, the total time including the sort is in $O(n \log n)$

Lab Exercise

- On Moodle

Thank you..!!